JAVASCRIPT — PARTE 4, 5 Y 6

Pedidos, cola de pedidos, carrito con deshacer e informe final

PARTE 4. PEDIDOS

En esta parte trabajamos con pedidos de clientes. Aprenderemos a convertir un texto en un objeto, comprobar si hay stock suficiente, calcular precios, actualizar el inventario y generar un ticket de compra.

Los conceptos principales que utilizaremos son:

- split(): divide un texto en partes.
- map(): transforma los elementos de un array.
- find(): busca un elemento que cumpla una condición.
- every(): comprueba si todos los elementos cumplen una condición.
- reduce(): acumula valores para obtener un resultado.
- unshift() y push(): añaden elementos a un array.
- join(): une elementos en un texto.
- El operador ...: permite copiar las propiedades de un objeto.

EJERCICIO 4.1. Convertir un texto en un pedido

Objetivo

Crear una función que reciba un texto con el nombre del cliente y los productos que ha pedido, y lo transforme en un objeto organizado.

Texto de entrada:

'Lucía|Tocadiscos:1;Vinilo Jazz:2'

Resultado esperado:

```
{
 cliente: 'Lucía',
 lineas: [
  { nombre: 'Tocadiscos', cantidad: 1 },
  { nombre: 'Vinilo Jazz', cantidad: 2 }
 ]
```

}

Código

```
export const parsearPedido = (texto) => {
 const partes = texto.split('|');
 const cliente = partes[0];

 const lineas = partes[1].split(';').map((linea) => {
  const [nombre, cantidad] = linea.split(':');

  return {
   nombre: nombre,
   cantidad: Number(cantidad)
  };
 });

 return {
  cliente,
  lineas
 };
};
```

Explicación paso a paso

1. Dividir el texto por el carácter |

```
const partes = texto.split('|');
```

El método split('|') separa el texto cada vez que encuentra el carácter |.

El resultado es:

```
[
 'Lucía',
 'Tocadiscos:1;Vinilo Jazz:2'
```

]

2. Obtener el nombre del cliente

const cliente = partes[0];

Los arrays empiezan a contar sus posiciones desde cero. Por tanto, partes[0] contiene 'Lucía'.

3. Separar los productos

const lineas = partes[1].split(';');

El método split(';') divide la segunda parte del texto por el punto y coma.

Resultado:

[

'Tocadiscos:1',

'Vinilo Jazz:2'

]

4. Transformar cada producto con map()

.map((linea) => {

map() recorre todos los elementos del array y crea un nuevo array con los resultados de la transformación.

5. Separar el nombre y la cantidad

const [nombre, cantidad] = linea.split(':');

Esta instrucción divide cada producto por los dos puntos y guarda sus partes en dos variables.

Por ejemplo, con 'Tocadiscos:1' obtenemos:

- nombre: 'Tocadiscos'
- cantidad: '1'

La cantidad sigue siendo un texto, por lo que hay que convertirla en número.

6. Convertir la cantidad a número

cantidad: Number(cantidad)

Number() convierte el texto '1' en el número 1.

7. Devolver el pedido completo

```
return {
 cliente,
 lineas
};
```

Se devuelve un objeto con el nombre del cliente y un array que contiene las líneas del pedido.

Sintaxis importante

- split(separador): divide un texto en un array.
- map((elemento) => ...): transforma cada elemento de un array.
- const [a, b] = array: extrae valores de un array mediante desestructuración.
- Number(valor): convierte un valor a número.
- { cliente, lineas }: es una forma abreviada de escribir { cliente: cliente, lineas: lineas }.

EJERCICIO 4.2. Comprobar si un pedido puede servirse

Objetivo

Crear una función que devuelva true si todos los productos del pedido existen en el catálogo y tienen stock suficiente. Si algún producto no existe o no hay suficientes unidades, debe devolver false.

Código

```
export const puedeServirse = (catalogo, pedido) => {
 return pedido.lineas.every((linea) => {
  const producto = catalogo.find(
   (producto) => producto.nombre === linea.nombre
  );

  return producto !== undefined &&
      producto.stock >= linea.cantidad;
 });
```

};

Explicación paso a paso

1. Recorrer todas las líneas del pedido

pedido.lineas.every((linea) => {

El método every() comprueba si todos los elementos de un array cumplen una condición.

- Si todas las líneas son válidas, devuelve true.
- Si al menos una línea no es válida, devuelve false.

2. Buscar el producto en el catálogo

const producto = catalogo.find(

(producto) => producto.nombre === linea.nombre

);

El método find() busca el primer producto cuyo nombre coincide con el nombre de la línea del pedido.

Si lo encuentra, devuelve el objeto correspondiente. Si no lo encuentra, devuelve undefined.

3. Comprobar que el producto existe y tiene stock suficiente

return producto !== undefined &&

producto.stock >= linea.cantidad;

Se comprueban dos condiciones:

- producto !== undefined: el producto existe en el catálogo.
- producto.stock >= linea.cantidad: hay suficientes unidades disponibles.

El operador && significa «y». Ambas condiciones deben cumplirse.

Ejemplo

Si el catálogo contiene un tocadiscos con un stock de 3 unidades y el pedido solicita 1, esa línea es válida.

Si el pedido solicita 5 unidades, la función devuelve false porque no hay stock suficiente.

Sintaxis importante

- every(): comprueba si todos los elementos cumplen una condición.

- find(): devuelve el primer elemento que cumple una condición.
- undefined: indica que no se ha encontrado un valor.
- >=: significa «mayor o igual que».
- &&: exige que se cumplan las dos condiciones.

EJERCICIO 4.3. Calcular el importe total de un pedido

Objetivo

Calcular cuánto cuesta un pedido sumando el precio de cada producto multiplicado por las unidades solicitadas.

Código

```
export const totalPedido = (catalogo, pedido) => {
 return pedido.lineas.reduce((total, linea) => {
  const producto = catalogo.find(
   (producto) => producto.nombre === linea.nombre
  );

  return total + producto.precio * linea.cantidad;
 }, 0);
};
```

Explicación paso a paso

1. Utilizar reduce() para acumular el importe

pedido.lineas.reduce((total, linea) => {

reduce() recorre las líneas del pedido y acumula un único resultado: el importe total.

En este caso:

- total: el importe acumulado hasta ese momento.
- linea: la línea del pedido que se está procesando.

2. Buscar el producto

const producto = catalogo.find(

(producto) => producto.nombre === linea.nombre

);

Buscamos el producto para conocer su precio unitario.

3. Calcular el importe de cada línea y sumarlo

return total + producto.precio * linea.cantidad;

Multiplicamos el precio por la cantidad pedida y añadimos el resultado al acumulador.

4. Inicializar el acumulador a cero

}, 0);

El 0 es el valor inicial de total. De esta forma, empezamos a sumar desde cero.

Ejemplo

Supongamos que el catálogo tiene estos precios:

- Tocadiscos: 200 €.
- Vinilo Jazz: 30 €.

El pedido contiene:

- 1 tocadiscos: 200 × 1 = 200 €.
- 2 vinilos: 30 × 2 = 60 €.

El resultado es:

260

Sintaxis importante

- reduce((acumulador, elemento) => ..., valorInicial): permite acumular un resultado.
- *: multiplicación.
- +: suma.
- producto.precio: accede a la propiedad precio del objeto.

Importante: esta función presupone que todos los productos del pedido existen en el catálogo. Por eso, normalmente conviene comprobar primero que el pedido puede servirse.

EJERCICIO 4.4. Servir un pedido sin modificar el catálogo original

Objetivo

Crear un catálogo nuevo en el que se reste del stock la cantidad solicitada de cada producto, manteniendo intacto el catálogo original.

Código

```
export const servirPedido = (catalogo, pedido) => {
 return catalogo.map((producto) => {
  const linea = pedido.lineas.find(
   (linea) => linea.nombre === producto.nombre
  );

  if (linea) {
   return {
    ...producto,
    stock: producto.stock - linea.cantidad
   };
  }

  return producto;
 });
};
```

Explicación paso a paso

1. Recorrer el catálogo con map()

return catalogo.map((producto) => {

map() permite crear un nuevo array a partir de los productos del catálogo.

2. Comprobar si el producto aparece en el pedido

const linea = pedido.lineas.find(

(linea) => linea.nombre === producto.nombre

);

Buscamos una línea del pedido cuyo nombre coincida con el producto del catálogo.

Si existe, find() devuelve esa línea. Si no existe, devuelve undefined.

3. Comprobar si se ha encontrado una línea

if (linea) {

Si linea existe, entramos en el bloque if y actualizamos el stock.

4. Crear una copia del producto con el stock actualizado

return {

...producto,

stock: producto.stock - linea.cantidad

};

El operador ...producto copia las propiedades del objeto original.

Después, se define la propiedad stock con el nuevo valor. Si ya existía, esta propiedad queda sobrescrita.

Por ejemplo, si el stock era 5 y se venden 2 unidades, el nuevo stock será 3.

5. Devolver los productos que no forman parte del pedido

return producto;

Si no hay ninguna línea coincidente, el producto se devuelve sin cambios.

¿Por qué no se modifica el catálogo original?

map() crea un array nuevo y, cuando se actualiza un producto, se utiliza un objeto nuevo mediante { ...producto }.

Esto permite mantener el catálogo original sin cambios para los productos que se actualizan.

Sintaxis importante

- map(): crea un array transformado.
- find(): busca un elemento.
- if (linea): ejecuta el bloque si linea existe y es un valor verdadero.
- ...producto: copia las propiedades enumerables del objeto.

- stock: producto.stock - linea.cantidad: calcula el nuevo stock.

Importante: la función no comprueba por sí misma si hay stock suficiente. Esa comprobación corresponde al ejercicio 4.2.

EJERCICIO 4.5. Generar el ticket de un pedido

Objetivo

Crear un único texto que contenga el nombre del cliente, el importe de cada línea y el total del pedido.

Resultado esperado:

Cliente: Lucía

1 x Tocadiscos = 200 €

2 x Vinilo Jazz = 60 €

TOTAL: 260 €

Código

```
export const generarTicket = (catalogo, pedido) => {
 const lineas = pedido.lineas.map((linea) => {
  const producto = catalogo.find(
   (producto) => producto.nombre === linea.nombre
  );

  const importe = producto.precio * linea.cantidad;

  return `${linea.cantidad} x ${linea.nombre} = ${importe} €`;
 });

 lineas.unshift(`Cliente: ${pedido.cliente}`);

 lineas.push(`TOTAL: ${totalPedido(catalogo, pedido)} €`);
```

```
  return lineas.join('\n');
};
```

Explicación paso a paso

1. Crear una línea de texto por producto

```
const lineas = pedido.lineas.map((linea) => {
```

Recorremos todas las líneas del pedido y generamos un texto para cada una.

2. Buscar el producto en el catálogo

```
const producto = catalogo.find(
  (producto) => producto.nombre === linea.nombre
);
```

Necesitamos conocer su precio para calcular el subtotal.

3. Calcular el importe de la línea

```
const importe = producto.precio * linea.cantidad;
```

Multiplicamos el precio unitario por la cantidad solicitada.

4. Crear el texto del producto

```
return `${linea.cantidad} x ${linea.nombre} = ${importe} €`;
```

Las comillas invertidas permiten utilizar una plantilla de texto o *template literal*.

Las expresiones escritas entre ${...} se sustituyen por sus valores.

Por ejemplo:

```
`2 x Vinilo Jazz = 60 €`
```

5. Añadir el nombre del cliente al principio

```
lineas.unshift(`Cliente: ${pedido.cliente}`);
```

unshift() añade un elemento al principio del array.

6. Añadir el total al final

```
lineas.push(`TOTAL: ${totalPedido(catalogo, pedido)} €`);
```

push() añade un elemento al final del array.

Se reutiliza la función totalPedido() del ejercicio 4.3.

7. Convertir el array en un único texto

return lineas.join('\n');

join('\n') une todas las cadenas utilizando un salto de línea entre ellas.

Sintaxis importante

- unshift(): añade elementos al principio de un array.
- push(): añade elementos al final.
- join('\n'): une las cadenas separándolas por saltos de línea.
- ${variable} : inserta el valor de una variable en un texto.

PARTE 5. COLA DE PEDIDOS Y CARRITO CON DESHACER

En esta parte trabajamos con estructuras de datos y métodos que modifican directamente los arrays recibidos.

Hay dos conceptos fundamentales:

- **Cola (FIFO):** el primero que entra es el primero que sale.
- **Pila (LIFO):** la última acción registrada es la primera que se deshace.

Los métodos principales son shift(), unshift(), push(), pop(), indexOf(), lastIndexOf() y splice().

EJERCICIO 5.1. Atender el siguiente pedido

Objetivo

Extraer y devolver el primer pedido de una cola.

Código

```
export const atenderSiguiente = (cola) => {
  return cola.shift();
};
```

Explicación

return cola.shift();

shift() elimina el primer elemento del array y lo devuelve.

Por ejemplo:

const cola = ['Pedido A', 'Pedido B', 'Pedido C'];

const pedido = atenderSiguiente(cola);

Resultado:

- pedido: 'Pedido A'.
- cola: ['Pedido B', 'Pedido C'].

La cola se modifica porque shift() elimina un elemento del array original.

Sintaxis importante

- shift(): elimina y devuelve el primer elemento.
- return: devuelve el resultado de la función.

Este comportamiento corresponde a una cola FIFO (*First In, First Out*).

EJERCICIO 5.2. Añadir un pedido urgente al principio de la cola

Objetivo

Insertar un pedido al principio de la cola y devolver la nueva longitud del array.

Código

```
export const agregarUrgente = (cola, pedido) => {
  return cola.unshift(pedido);
};
```

Explicación

return cola.unshift(pedido);

unshift() añade el pedido al principio de la cola y devuelve la nueva longitud del array.

Ejemplo:

const cola = ['Pedido A', 'Pedido B'];

const longitud = agregarUrgente(cola, 'Pedido urgente');

Resultado:

- cola: ['Pedido urgente', 'Pedido A', 'Pedido B'].

- longitud: 3.

Sintaxis importante

- unshift(elemento): inserta un elemento al principio.
- El valor devuelto por unshift() es la longitud del array, no el elemento añadido.

EJERCICIO 5.3. Añadir un producto al carrito y registrarlo en el historial

Objetivo

Añadir el nombre de un producto al final del carrito y registrar la operación en el historial.

Código

```
export const agregarAlCarrito = (carrito, historial, nombre) => {
  carrito.push(nombre);


  historial.push({
    accion: 'agregar',
    nombre: nombre
  });
};
```

Explicación paso a paso

1. Añadir el producto al carrito

```
carrito.push(nombre);
```

push() añade el nombre del producto al final del array.

2. Registrar la acción en el historial

```
historial.push({
  accion: 'agregar',
  nombre: nombre
});
```

Se añade al historial un objeto que contiene:

- accion: indica qué operación se ha realizado.
- nombre: indica el producto afectado.

Por ejemplo, al añadir 'Vinilo Jazz', se registra:

```
{
 accion: 'agregar',
 nombre: 'Vinilo Jazz'
}
```

El historial permitirá deshacer posteriormente la operación.

Sintaxis importante

- push(): añade un elemento al final del array.
- { accion: 'agregar', nombre: nombre }: crea un objeto que registra la operación.
- carrito e historial se modifican directamente.

La función no necesita un return, porque el ejercicio solo pide realizar las modificaciones.

EJERCICIO 5.4. Quitar un producto del carrito

Objetivo

Eliminar la primera aparición de un nombre en el carrito y registrar la acción en el historial.

La función debe devolver:

- true si encuentra y elimina el producto.
- false si el producto no está en el carrito.

Código

```
export const quitarDelCarrito = (carrito, historial, nombre) => {
 const posicion = carrito.indexOf(nombre);

 if (posicion === -1) {
  return false;
```

```
 }

 carrito.splice(posicion, 1);

 historial.push({
  accion: 'quitar',
  nombre: nombre,
  posicion: posicion
 });

 return true;
};
```

Explicación paso a paso

1. Buscar la posición del producto

```
const posicion = carrito.indexOf(nombre);
```

indexOf() devuelve la posición de la primera aparición del elemento.

Si no lo encuentra, devuelve -1.

2. Comprobar si el producto existe

```
if (posicion === -1) {
 return false;
}
```

Si la posición es -1, el producto no está en el carrito.

La función devuelve false sin modificar el carrito ni el historial.

3. Eliminar el producto

```
carrito.splice(posicion, 1);
```

splice(posicion, 1) elimina un elemento a partir de la posición indicada.

El primer argumento es la posición y el segundo indica cuántos elementos se eliminan.

4. Registrar la operación

```
historial.push({
  accion: 'quitar',
  nombre: nombre,
  posicion: posicion
});
```

Se guarda la acción, el nombre y la posición original del producto.

Guardar la posición es importante para poder restaurar el carrito exactamente como estaba.

5. Devolver true

```
return true;
```

Se devuelve true porque el producto se ha encontrado y eliminado.

Sintaxis importante

- indexOf(elemento): devuelve la posición de la primera aparición.
- ===: compara valores y tipos.
- splice(posicion, cantidad): elimina o inserta elementos.
- return false: finaliza la función devolviendo false.
- return true: indica que la operación se ha realizado.

EJERCICIO 5.5. Deshacer la última acción del carrito

Objetivo

Deshacer la última operación registrada en el historial.

Hay dos posibilidades:

1. Si la acción fue 'agregar', se elimina del carrito la última aparición del producto.
2. Si la acción fue 'quitar', se vuelve a insertar el producto en su posición original.

La función devuelve false si el historial está vacío y true si se deshace una acción reconocida.

Código

```
export const deshacer = (carrito, historial) => {
 if (historial.length === 0) {
  return false;
 }

 const accion = historial.pop();

 if (accion.accion === 'agregar') {
  const posicion = carrito.lastIndexOf(accion.nombre);

  if (posicion !== -1) {
   carrito.splice(posicion, 1);
  }

  return true;
 }

 if (accion.accion === 'quitar') {
  carrito.splice(accion.posicion, 0, accion.nombre);

  return true;
 }

 return false;
};
```

Explicación paso a paso

1. Comprobar si el historial está vacío

```
if (historial.length === 0) {
  return false;
}
```

Si no hay acciones registradas, no hay nada que deshacer.

2. Extraer la última acción

```
const accion = historial.pop();
```

pop() elimina y devuelve el último elemento del array.

Esto permite aplicar el comportamiento LIFO (*Last In, First Out*): la última acción realizada es la primera que se deshace.

3. Deshacer una acción de tipo 'agregar'

```
if (accion.accion === 'agregar') {
```

Si la última acción fue añadir un producto, debemos quitarlo del carrito.

```
const posicion = carrito.lastIndexOf(accion.nombre);
```

lastIndexOf() devuelve la posición de la última aparición del nombre.

Se utiliza en lugar de indexOf() porque puede haber varios productos con el mismo nombre y queremos eliminar la última aparición.

```
if (posicion !== -1) {
  carrito.splice(posicion, 1);
}
```

Si el producto se encuentra, se elimina una unidad.

Después, la función devuelve true.

4. Deshacer una acción de tipo 'quitar'

```
if (accion.accion === 'quitar') {
  carrito.splice(accion.posicion, 0, accion.nombre);

  return true;
}
```

Aquí utilizamos splice() de una forma diferente.

Su sintaxis es:

array.splice(posicion, cantidadAEliminar, elementoAInsertar);

En este caso:

- accion.posicion: posición original del producto.
- 0: no se elimina ningún elemento.
- accion.nombre: producto que queremos insertar.

Así, el producto vuelve a su posición original.

5. Gestionar acciones no reconocidas

return false;

Si el objeto extraído no corresponde a ninguna de las dos acciones previstas, la función devuelve false.

Sintaxis importante

- pop(): elimina y devuelve el último elemento.
- lastIndexOf(elemento): encuentra la última aparición.
- splice(posicion, 1): elimina un elemento.
- splice(posicion, 0, elemento): inserta un elemento sin eliminar ninguno.
- historial.length: indica cuántas acciones están registradas.

Ejemplo de funcionamiento

Estado inicial:

const carrito = ['Tocadiscos', 'Vinilo Jazz'];

const historial = [];

Añadimos un producto:

agregarAlCarrito(carrito, historial, 'Auriculares');

El carrito pasa a ser:

['Tocadiscos', 'Vinilo Jazz', 'Auriculares']

El historial contiene:

[

{ accion: 'agregar', nombre: 'Auriculares' }

]

Si ejecutamos:

deshacer(carrito, historial);

El carrito vuelve a ser:

['Tocadiscos', 'Vinilo Jazz']

Y el historial queda vacío.

PARTE 6. INFORME FINAL

En esta última parte combinamos las funciones anteriores para procesar todos los pedidos, identificar los productos vendidos y generar un gráfico textual del stock.

Los conceptos principales son:

- Bucles while.
- Condicionales if / else.
- Arrays y objetos.
- Set para eliminar duplicados.
- flatMap() para transformar y aplanar arrays.
- sort() para ordenar.
- new Array() y fill() para generar barras de stock.

EJERCICIO 6.1. Procesar toda la cola de pedidos

Objetivo

Atender todos los pedidos de una cola, uno por uno.

- Si un pedido puede servirse, se actualiza el catálogo y se guarda en servidos.
- Si no puede servirse, se guarda en rechazados.
- Al finalizar, la cola debe quedar vacía.

La función devuelve un objeto con tres propiedades:

```
{
  catalogo,
  servidos,
```

```
 rechazados
}
```

Código

```
export const procesarCola = (catalogo, cola) => {
 const servidos = [];
 const rechazados = [];

 while (cola.length > 0) {
  const pedido = atenderSiguiente(cola);

  if (puedeServirse(catalogo, pedido)) {
   catalogo = servirPedido(catalogo, pedido);
   servidos.push(pedido);
  } else {
   rechazados.push(pedido);
  }
 }

 return { catalogo, servidos, rechazados };
};
```

Explicación paso a paso

1. Crear los arrays de resultados

```
const servidos = [];
const rechazados = [];
```

Creamos dos arrays vacíos para clasificar los pedidos según su resultado.

2. Repetir mientras queden pedidos

```
while (cola.length > 0) {
```

El bucle while se ejecuta mientras su condición sea verdadera.

En este caso, continúa mientras la cola tenga al menos un pedido.

3. Extraer el siguiente pedido

const pedido = atenderSiguiente(cola);

Se utiliza la función del ejercicio 5.1, que elimina y devuelve el primer pedido.

4. Comprobar si puede servirse

if (puedeServirse(catalogo, pedido)) {

Se utiliza la función del ejercicio 4.2 para comprobar si existen todos los productos y hay stock suficiente.

5. Actualizar el catálogo y guardar el pedido

catalogo = servirPedido(catalogo, pedido);

servidos.push(pedido);

Si el pedido es válido, se crea un catálogo actualizado y se registra el pedido como servido.

6. Registrar los pedidos rechazados

else {

rechazados.push(pedido);

}

Si el pedido no puede servirse, se guarda en rechazados. El catálogo no se modifica por ese pedido.

7. Devolver el informe

return { catalogo, servidos, rechazados };

Al finalizar el bucle, se devuelve el catálogo actualizado y los dos arrays de pedidos.

Sintaxis importante

- while (condicion): repite un bloque mientras se cumpla una condición.
- if / else: permite ejecutar instrucciones diferentes según una condición.
- push(): añade un pedido al array correspondiente.
- cola.length > 0: comprueba si la cola contiene elementos.
- { catalogo, servidos, rechazados }: devuelve un objeto con tres propiedades.

Detalle importante

El catálogo se actualiza después de cada pedido aceptado. Por eso, si dos pedidos necesitan el mismo producto y el primero consume parte del stock, el segundo se comprueba utilizando el stock restante.

Además, la cola original queda vacía porque atenderSiguiente() utiliza shift().

EJERCICIO 6.2. Obtener los productos vendidos sin repetirlos

Objetivo

Recibir un array de pedidos y devolver los nombres de todos los productos vendidos, sin duplicados y ordenados alfabéticamente.

Código

```
export const productosVendidos = (pedidos) => {
 return [...new Set(
  pedidos.flatMap((pedido) =>
   pedido.lineas.map((linea) => linea.nombre)
  )
 )].sort();
};
```

Explicación paso a paso

1. Recorrer todos los pedidos y extraer sus productos

```
pedidos.flatMap((pedido) =>
 pedido.lineas.map((linea) => linea.nombre)
)
```

Primero, map() obtiene los nombres de los productos de cada pedido.

Después, flatMap() combina los resultados y aplana un nivel de arrays.

Por ejemplo, el resultado podría ser:

```
[
 'Tocadiscos',
 'Vinilo Jazz',
```

'Vinilo Jazz',

'Altavoz'

]

2. Eliminar los duplicados con Set

new Set(...)

Un Set es una estructura que almacena valores únicos. Si un producto aparece varias veces, solo conserva una aparición.

3. Convertir el Set en un array

[...new Set(...)]

El operador ... permite expandir los elementos del Set dentro de un array nuevo.

4. Ordenar alfabéticamente

.sort()

sort() ordena los elementos del array.

En el caso de nombres de productos formados por texto, permite obtener una lista alfabética básica.

Ejemplo

Entrada:

```
[
 {
  cliente: 'Lucía',
  lineas: [
   { nombre: 'Tocadiscos', cantidad: 1 },
   { nombre: 'Vinilo Jazz', cantidad: 2 }
  ]
 },
 {
  cliente: 'Pablo',
  lineas: [
   { nombre: 'Vinilo Jazz', cantidad: 1 },
```

```
    { nombre: 'Altavoz', cantidad: 1 }
  ]
 }
]
```

Resultado:

```
[
 'Altavoz',
 'Tocadiscos',
 'Vinilo Jazz'
]
```

Sintaxis importante

- flatMap(): transforma los elementos y aplana un nivel del resultado.
- Set: almacena valores únicos.
- [...iterable]: convierte los elementos de una estructura iterable en un array.
- sort(): ordena los elementos de un array.

Nota: sort() sin parámetros utiliza el orden de cadenas de JavaScript. Para una ordenación alfabética adaptada a las reglas del español, se podría utilizar localeCompare() con la configuración regional correspondiente.

EJERCICIO 6.3. Crear un gráfico textual del stock

Objetivo

Generar un array de textos que represente las unidades disponibles de cada producto mediante cuadrados negros.

Por ejemplo:

Altavoz: ■■■ (3)

La barra debe construirse obligatoriamente con new Array(...).fill('■').

Código

```
export const graficoStock = (catalogo) => {
 return catalogo.map((producto) =>
```

```
  `${producto.nombre}: ${new Array(producto.stock)
    .fill('■')
    .join('')} (${producto.stock})`
 );
};
```

Explicación paso a paso

1. Recorrer el catálogo

return catalogo.map((producto) =>

map() crea un texto para cada producto del catálogo.

2. Crear un array con tantas posiciones como unidades haya en stock

new Array(producto.stock)

Si producto.stock vale 3, se crea un array de longitud 3.

3. Rellenar el array con cuadrados negros

.fill('■')

fill('■') rellena todas las posiciones del array con el carácter ■.

Si el stock es 3, el array queda formado por tres cuadrados.

4. Unir los cuadrados en una cadena

.join('')

join('') une los elementos sin añadir ningún separador entre ellos.

El resultado es:

■■■

5. Crear el texto final

`${producto.nombre}: ${...} (${producto.stock})`

La plantilla incluye:

- El nombre del producto.
- Dos puntos y un espacio.
- La barra de cuadrados.
- El número de unidades entre paréntesis.

Ejemplo

Si el catálogo contiene:

```
[
 { nombre: 'Altavoz', stock: 3 },
 { nombre: 'Tocadiscos', stock: 1 },
 { nombre: 'Vinilo Jazz', stock: 0 }
]
```

El resultado será:

```
[
 'Altavoz: ■■■ (3)',
 'Tocadiscos: ■ (1)',
 'Vinilo Jazz:  (0)'
]
```

Cuando el stock es cero, no se genera ningún cuadrado porque el array tiene longitud cero.

Sintaxis importante

- new Array(n): crea un array de longitud n.
- fill(valor): rellena el array con un valor.
- join(''): une los elementos sin separador.
- map(): transforma cada producto en un texto.

Importante: esta solución presupone que el stock es un número entero no negativo. new Array() puede producir un error si recibe una longitud inválida.

RESUMEN GENERAL DE LOS MÉTODOS

Método o recurso	**¿Para qué sirve?**
split()	Divide un texto en partes.
map()	Transforma cada elemento de un array.
find()	Busca el primer elemento que cumple una condición.

Método o recurso	¿Para qué sirve?
every()	Comprueba si todos los elementos cumplen una condición.
reduce()	Acumula valores para obtener un resultado.
flatMap()	Transforma elementos y aplana un nivel de arrays.
push()	Añade un elemento al final.
pop()	Elimina y devuelve el último elemento.
shift()	Elimina y devuelve el primer elemento.
unshift()	Añade un elemento al principio.
indexOf()	Busca la primera aparición de un elemento.
lastIndexOf()	Busca la última aparición de un elemento.
splice()	Elimina, inserta o sustituye elementos.
join()	Une elementos de un array en un texto.
sort()	Ordena los elementos de un array.
Set	Almacena valores únicos.
Number()	Convierte un valor a número.
new Array(n)	Crea un array con una longitud determinada.
fill()	Rellena las posiciones de un array.
...	Expande elementos o copia propiedades, según el contexto.
return	Devuelve un resultado y termina la ejecución de la función.
if / else	Permite tomar decisiones.
while	Repite instrucciones mientras una condición sea verdadera.

DIFERENCIAS QUE DEBES RECORDAR

1. Métodos que modifican el array original

Los siguientes métodos modifican directamente el array sobre el que se ejecutan:

- push()

- pop()
- shift()
- unshift()
- splice()
- sort()
- fill()

2. Métodos que generan resultados nuevos

Estos métodos permiten crear arrays nuevos o calcular resultados sin modificar directamente el array original:

- map()
- filter()
- flatMap()
- reduce()
- slice()

find(), every(), indexOf() y lastIndexOf() buscan o comprueban información sin modificar el array.

join() devuelve una cadena de texto.

Set elimina duplicados de su propia colección, y el operador ... permite expandir sus elementos dentro de un array nuevo.

3. Diferencia entre cola y pila

Cola FIFO (primero en entrar, primero en salir):

- shift(): extrae el primer elemento.
- unshift(): inserta un elemento al principio.

Pila LIFO (último en entrar, primero en salir):

- push(): añade una acción al historial.
- pop(): extrae la última acción para deshacerla.

4. Diferencia entre find(), every() y reduce()

- find() devuelve un elemento que cumple una condición, o undefined si no lo encuentra.

- every() devuelve un booleano: true si todos los elementos cumplen la condición y false en caso contrario.
- reduce() combina los elementos para obtener un único resultado, como el importe total de un pedido.

CONCLUSIÓN

Los ejercicios de estas tres partes enseñan a trabajar con pedidos, inventarios, arrays y objetos en JavaScript.

La idea fundamental es elegir el método adecuado según el objetivo:

- Para **buscar**, utilizamos find().
- Para **comprobar todas las condiciones**, utilizamos every().
- Para **sumar importes**, utilizamos reduce().
- Para **transformar datos**, utilizamos map().
- Para **eliminar duplicados**, utilizamos Set.
- Para **gestionar una cola**, utilizamos shift() y unshift().
- Para **deshacer acciones**, utilizamos un historial con pop() y splice().
- Para **generar textos**, utilizamos plantillas literales y join().

Comprender qué devuelve cada método, si modifica el array original y cómo se combinan los métodos es esencial para resolver correctamente este tipo de ejercicios.